CMSC6950 — Fall 2025
Homework #7

Due Date: Work must be submitted to GradeScope (for all problems) by 5pm (St. John's time) on Thursday, December 4.

1. (10 points) Write two pieces of code that generate values of `nsample+1` points, x, evenly spaced between 0 and 1 (including the endpoints) and values of the function $f(x) = \sin(x^2)$ at each of those points. In the first piece of code, you should store the values for x and $f(x)$ in Python lists, using `.append` to add each value to its list within a `for` loop. For the second piece of code, you should use native `NumPy` routines to generate an ndarray of points and a second ndarray of function values.

   Run your code for several values of `nsample`, ranging over several orders of magnitude in size. Time the two versions of the code for each size and discuss the time comparison between the list-based and NumPy versions of the code. To time a section of code, you can use the `datetime` module, writing

   ```
   start = datetime.now()
   # Code to time
   end = datetime.now()
   elapsed = (end-start).total_seconds()
   ```

   after importing the module using `from datetime import datetime`. You should turn in a listing of your code and its output and some commentary on the relative timings of the two approaches.

2. (10 points) Use `scipy.special` to plot the first 5 Chebyshev polynomials of the first kind on the interval $[-1, 1]$ and their zeros on the same graph.

3. (15 points) Consider the function $f(x) = e^{\sin(\pi x)}$

   (a) Use `interp1d` from `scipy.interpolate` to interpolate $f(x)$ for $0 \leq x \leq 3$ using data at 15 evenly spaced points in 3 different ways, presenting the 3 interpolants (sampled at many points) on the same graph.

   (b) Use `CubicSpline` from `scipy.interpolate` to interpolate $f(x)$ for $0 \leq x \leq 3$ using data at 15 evenly spaced points, and plot the derivative of the cubic spline interpolant and the exact value of $f'(x)$ (sampled at many points) on the same graph.

   (c) Use `fixed_quad` from `scipy.interpolate` to approximate $\int_0^3 f(x)dx$ for "orders" 1 through 20. How do these answers compare to those generated by the adaptive routines in `quad` and `quadrature`?

4. (15 points) Consider the function $f(x, y) = (10000xy - 1)^2 + (e^{-x} + e^{-y} - 1.0001)^2$. Try to find the minimum value of this function using:

   (a) BFGS without providing the Jacobian

   (b) BFGS with providing the Jacobian

   (c) Newton-CG with providing the Jacobian but not the Hessian

(d) Newton-CG with providing both the Jacobian and Hessian

(e) Nelder-Mead

each with the initial guess $(x, y) = (0, 1)$. For each method, you may need to experiment with the tolerance that you set for convergence (the `tol` kwarg for `minimize` from `scipy.optimize`) or the parameters of the method (such as `maxfev` for Nelder-Mead, set by passing a dictionary to the `options` kwarg). What is the best minimizer that you can find?